

MEDHAVA

Requirements registry

One row per capability, the rung it has reached, and the thing that proves it.

This exists because the honest answer to “how much of this works” used to live in a sentence rather than in the repository. Twice in one session that sentence was wrong in a way nothing could contradict — a claim that there was no continuous integration while the workflow file sat in the tree, and an archive reported at a size and file count that were both read off the wrong line. A row here is a claim with a receipt attached, and `node brand/site/checkregistry.js` refuses the claim when the receipt is missing.

Nothing in this document is typed. Every status, path, command and count is read from `brand/site/registry.js`, `brand/site/modules.js`, `brand/site/built.js` and `docs/verification/EVIDENCE.md` when it is generated. Regenerate with `node brand/delivery/website/mkregistry.js`.

Where the project actually stands

Rung	Rows
NOT STARTED	6
SPECIFIED	98
IMPLEMENTED	1
TESTED	26
BLOCKED	2
Total	133

No row sits at DESIGNED, VERIFIED, PRODUCTION-READY, DEPRECATED. VERIFIED and PRODUCTION-READY are empty **by rule** — the gate refuses either one, because neither can be earned from inside a repository that has never been deployed or checked against anything outside itself.

Of 113 apps, **19 have a recorded passing test** and 0 are implemented without one. The remaining 94 are specified: written down in full, and not standing up.

What each rung costs to claim

NOT STARTED — Nothing exists. A row here may cite no file at all — a status carrying a path is not "not started", and the gate refuses the pair.

SPECIFIED — Written down in a register somebody can read. Every app in the module list is at least this, which is why it is the floor and not an achievement.

DESIGNED — Specified, and the decision is argued somewhere with what would make it wrong — an architecture note, a stack layer, or a rule carrying its "never".

IMPLEMENTED — Code exists and runs. Tests may exist; they are not in the gated suite, so nothing would notice if they broke.

TESTED — An automated test drives it, passes, runs inside `npm test`, and that run is recorded in `docs/verification/EVIDENCE.md` with exit 0. Three conditions, all checked by a gate. "There is a test file" earns IMPLEMENTED, not this.

VERIFIED — Tested, and additionally checked against a source outside this repository. Nothing here is at this rung, and the gate refuses to let anything be put there. That is not modesty: the one time an engine in this project was checked against an outside document, the document found a defect that hundreds of internal checks had agreed with.

PRODUCTION-READY — Deployed, reachable and smoke-tested there. Nothing is here either, and nothing can be until something is actually deployed somewhere.

BLOCKED — Cannot proceed, and the row names by what. A blocker naming no obstacle is refused.

DEPRECATED — Was real, is being withdrawn.

The evidence behind every TESTED row

Each command below was run through `tools/evidence.js`, which records the exit code the process returned, the commit, whether the tree was dirty, and the SHA-256 of the files the run was about. `node tools/evidence.js --check` re-runs them all and reports where a result has moved.

Command	Recorded exit	Rows resting on it
<code>node brand/site/checkcoverage.js</code>	0	1
<code>node core/tests/core.test.js</code>	0	1
<code>node core/tests/live.test.js</code>	0	1
<code>node core/tests/packs.test.js</code>	0	1
<code>node core/tests/schema.test.js</code>	0	1
<code>node tools/evidence.test.js</code>	0	1
<code>npm run apps</code>	0	14
<code>npm run medhava</code>	0	4
<code>npm run selftest</code>	0	2

These are the commands a row in this document depends on. The log holds other runs besides — read [docs/verification/EVIDENCE.md](#) for all of them. Their number is left out of this document on purpose: it changes every time anything is recorded, and a generated file that goes stale as a side effect of using the tool it documents fails the build for a reason that has nothing to do with what it says.

Capabilities that are not apps

A registry of apps alone would report this project as far healthier than it is. It would never mention that nothing is deployed, that no outside integration is live, or that three whole product surfaces have not been started. Those sit in the same table, held to the same rung definitions.

CAP-ISOLATION · Tenant isolation in the database

TESTED

Files: `core/schema.postgres.sql` · `core/tests/live.test.js` · `medhava/test/isolation.test.js`

Proven by: `node core/tests/live.test.js` — recorded at exit 0.

The schema is loaded into a real Postgres and driven by a role that is neither superuser nor table owner — without that role the policies are inert and the test would pass while proving nothing.

CAP-SCHEMA · The production schema

TESTED

Files: `core/schema.postgres.sql` · `core/tests/schema.test.js`

Proven by: `node core/tests/schema.test.js` — recorded at exit 0.

151 tables that execute, every business table company-scoped, money as integer paise. Never loaded with production-scale data; nothing here is a performance claim.

CAP-PACKS · Trade configuration as data

TESTED

Files: `core/packs.js` · `core/tenant.js` · `core/tests/packs.test.js`

Proven by: `node core/tests/packs.test.js` — recorded at exit 0.

A pack renames and extends; it may not invent a concept or carry executable code. The effective-dated tenant overlay is checked in the same run.

CAP-GROUP · Multi-company consolidation

TESTED

Files: `core/tests/core.test.js`

Proven by: `node core/tests/core.test.js` — recorded at exit 0.

Posted across a 10 x 10 grid and then 11 x 11 with no code changed, so the arithmetic carries no ceiling of its own.

CAP-SHELL · Sign-in, session and the web shell

TESTED

Files: `medhava/server/auth.js` · `medhava/server/api.js` · `medhava/web/index.html` · `medhava/test/shell.test.js`

Proven by: `npm run medhava` — recorded at exit 0.

Sign-in, company switching and a screen that shows isolation refusing a cross-company read. Single server, no session store, no password reset.

CAP-CI · Continuous integration

IMPLEMENTED

Files: `.github/workflows/ci.yml`

Runs the suite on push. Not recorded here as TESTED because what proves a CI file is a run on the service, and this register only counts runs recorded in this repository. I claimed twice this session that there was no CI at all.

CAP-EVIDENCE · Verification evidence capture

TESTED

Files: `tools/evidence.js` · `tools/evidence.test.js` · `docs/verification/EVIDENCE.md`

Proven by: `node tools/evidence.test.js` — recorded at exit 0.

Records command, exit code, commit, dirty tree and artifact hashes. Tamper-evident, not tamper-proof: anyone with write access can edit the file. The test plants a drifted exit code and a hand-waved excuse and requires both to be refused — it began as two checks I ran by hand, which is a check that only holds while somebody remembers to run it.

CAP-DOCS · Generated, gated documentation

TESTED

Files: `brand/delivery/manifest.js` · `brand/site/checkcoverage.js`

Proven by: `node brand/site/checkcoverage.js` — recorded at exit 0.

Every delivered document is generated from a register and refuses to ship ungated. It documents a design; a gated document is not a working feature.

CAP-DEPLOY · Deployment to a production environment

NOT STARTED

Blocked by: No server, no domain and no credentials exist. DEPLOYMENT.md and deploy/ are a written runbook, which is a plan for deploying and not a deployment. Nothing has ever been installed anywhere from them.

CAP-PUBLISH · The static site published at a public URL

BLOCKED

Files: `brand/delivery/website/mksite.js` · `brand/site/checksite.js` · `.github/workflows/pages.yml`

This is deliberately its own row rather than folded into CAP-DEPLOY. They are different things: CAP-DEPLOY is the product server with its database, which cannot be deployed from this environment at all, and this is a static site that is built, verified and one setting away from being live.

Blocked by: Blocked on one repository setting, not on any code. The site is assembled by `mksite.js` and verified by `checksite.js`, which serves it over real HTTP and drives it in Chromium — 13 assertions, recorded. The workflow that would publish it cannot: creating a GitHub Pages site needs administration rights the Actions token does not have here, and the attempt fails with "Resource not accessible by integration". Pages has never been enabled on this repository — the runs from July failed the same way. Settings → Pages → Build and deployment → Source: GitHub Actions is the whole fix, and it is the owner's to make. `enablement: true` was expected to do it unattended and does not.

CAP-MONITOR · Monitoring, alerting and health checks

SPECIFIED

Blocked by: Nothing to monitor until something is deployed. Depends on CAP-DEPLOY.

CAP-INTEGRATIONS · Live outside integrations

BLOCKED

Blocked by: Every marketplace, courier, tax portal, bank feed and payment provider needs live credentials, and this repository must never hold one. This cannot be raised from inside the repository at all — it needs an environment holding secrets that no commit ever sees. Simulating one and calling it connected is the failure this whole register exists to make impossible.

CAP-AI · AI gateway, agent permissions and evaluation

SPECIFIED

Blocked by: The provider router is the one piece that runs. There is no gateway, no permission model for what an agent may do on a company's data, and no evaluation set, so no quality claim can be made about any answer the system gives.

CAP-ANALYTICS · Query engine and report builder on real data

SPECIFIED

Blocked by: The prototype report builder computes over an in-page store. There is no query engine against the database.

CAP-DEVPLATFORM · Developer platform — public API, webhooks, SDKs

NOT STARTED

Blocked by: Not begun. The internal API is four routes for two modules and is not a public surface: no versioning, no keys, no rate limiting, no webhook delivery.

CAP-STUDIO · Studio — building screens and flows without code

NOT STARTED

Blocked by: Not begun. Packs configure vocabulary and fields; they do not let anyone build a screen.

CAP-MARKETPLACE · Extension marketplace

NOT STARTED

Blocked by: Not begun, and it cannot begin before CAP-DEVPLATFORM: there is nothing for a third party to extend.

CAP-MOBILE · Mobile

SPECIFIED

Blocked by: The shell is responsive markup that has never been opened on a phone by any check here. A responsive claim nobody has tested is a claim, and this register will not count it as a result.

CAP-THREATMODEL · Threat model and adversarial security testing

NOT STARTED

Blocked by: Isolation is proven against an honest client. Nobody has written down who the attacker is, and no test attacks the system the way one would.

CAP-SCALE · Behaviour at production scale

NOT STARTED

Blocked by: Every test here runs on a handful of rows. No load test, no query plan reviewed, no index measured. Nothing in this repository supports any statement about speed under real volume.

Every app, by module

In the order the module list gives them. Re-ordering a sequence somebody specified is a second opinion nobody asked for.

Module 01 · Platform

8 app(s) · 2 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP - 01-01	Identity, Settings & Audit	SPECIFIED	—	Specified in the module register; not standing up.
APP - 01-02	Industry Packs	SPECIFIED	—	Specified in the module register; not standing up.
APP - 01-03	Ask & Print	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 01-04	Communications	SPECIFIED	—	Specified in the module register; not standing up.
APP - 01-05	WhatsApp Command Console	SPECIFIED	—	Specified in the module register; not standing up.
APP - 01-06	Data Privacy & Consent	SPECIFIED	—	Specified in the module register; not standing up.
APP - 01-07	Provider Router & Cost Guard	TESTED	npm run selftest	Fallback order, breaker and spend ceiling all exercised. No provider is actually connected: the selftest drives fakes, so this proves the rule, not the integration.
APP - 01-08	Payment Data Scope	SPECIFIED	—	Specified in the module register; not standing up.

Module 02 · Design & Sampling

2 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-02-01	PLM & Development	SPECIFIED	—	Specified in the module register; not standing up.
APP-02-02	Design / IP Register	SPECIFIED	—	Specified in the module register; not standing up.

Module 03 · Inventory & Catalog

4 app(s) · 1 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP - 03-01	Stock	TESTED	npm run medhava	Receipt, issue and transfer against the real database inside row-level security. Reorder alerts, batches, kits and dead-stock are specified and absent.
APP - 03-02	Catalog / PIM	SPECIFIED	—	Specified in the module register; not standing up.
APP - 03-03	Kit & Combo SKU	SPECIFIED	—	Specified in the module register; not standing up.
APP - 03-04	Master-Data Hygiene	SPECIFIED	—	Specified in the module register; not standing up.

Module 04 · CRM

4 app(s) · 3 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP - 04-01	CRM & Customer 360	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 04-02	Documents & eSign	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 04-03	Helpdesk & Live Chat	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 04-04	Forms & Feedback (NPS)	SPECIFIED	—	Specified in the module register; not standing up.

Module 05 · Sales

8 app(s) · 5 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP - 05-01	D2C Sales	TESTED	npm run medhava	One order posts lines, invoice, stock movement and ledger in a single transaction. No storefront connected — the order arrives over the API, not from a shop. A prototype browser screen of the same name exists separately.
APP - 05-02	B2B & Credit	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 05-03	Export	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 05-04	POS	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 05-05	Quotes & Proforma	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 05-06	Couriers & AWB	SPECIFIED	—	Specified in the module register; not standing up.
APP - 05-07	Subscriptions	SPECIFIED	—	Specified in the module register; not standing up.
APP - 05-08	Customisation & Made-to-Measure	SPECIFIED	—	Specified in the module register; not standing up.

Module 06 · Planning & Requirements (MRP)

3 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-06-01	Demand Forecast & Signal	SPECIFIED	—	Specified in the module register; not standing up.
APP-06-02	Requirement Explosion (MRP run)	SPECIFIED	—	Specified in the module register; not standing up.
APP-06-03	Open-to-Buy / Budget Ceiling	SPECIFIED	—	Specified in the module register; not standing up.

Module 07 · Purchase

3 app(s) · 2 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-07-01	Procurement	TESTED	npm run medhava	A purchase order raised against an active vendor, then received — in part or in full — against the real database inside row-level security. A short receipt adds only what arrived and owes only for that; the order stays open for the rest. What is NOT here: the vendor invoice, and so the full three-way match of R07.11, plus approval routing, landed cost and dated vendor pricing. Nine of module 07's twelve rules are specified and unenforced, and none of them is stubbed.
APP-07-02	Vendor Management	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside npm test — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP-07-03	Insurance Register	SPECIFIED	—	Specified in the module register; not standing up.

Module 08 · Manufacturing

4 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-08-01	Production Orders	SPECIFIED	—	Specified in the module register; not standing up.
APP-08-02	Piece-rate & Contractors	SPECIFIED	—	Specified in the module register; not standing up.
APP-08-03	BOM & Consumption	SPECIFIED	—	Specified in the module register; not standing up.
APP-08-04	Maintenance	SPECIFIED	—	Specified in the module register; not standing up.

Module 09 · Quality & Compliance

2 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-09-01	Quality Control	SPECIFIED	—	Specified in the module register; not standing up.
APP-09-02	Certificate & Compliance Register	SPECIFIED	—	Specified in the module register; not standing up.

Module 10 · Warehouse

3 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-10-01	Picking & Bins	SPECIFIED	—	Specified in the module register; not standing up.
APP-10-02	Barcode Operations	SPECIFIED	—	Specified in the module register; not standing up.
APP-10-03	Packing Video	SPECIFIED	—	Specified in the module register; not standing up.

Module 11 · Logistics

5 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-11-01	Rates & Zones	SPECIFIED	—	Specified in the module register; not standing up.
APP-11-02	NDR & RTO Rescue	SPECIFIED	—	Specified in the module register; not standing up.
APP-11-03	COD Remittance	SPECIFIED	—	Specified in the module register; not standing up.
APP-11-04	Handover & Manifest	SPECIFIED	—	Specified in the module register; not standing up.
APP-11-05	Fleet	SPECIFIED	—	Specified in the module register; not standing up.

Module 12 · Accounting & GST

9 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-12-01	Accounting	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-02	Invoicing	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-03	Expenses	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-04	GST & Tax	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-05	ITC Reconciliation	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-06	Receivables, Payables & PDC	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-07	Fixed Assets & Depreciation	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-08	Year-End Close & Period Lock	SPECIFIED	—	Specified in the module register; not standing up.
APP-12-09	Finance Reports	SPECIFIED	—	Specified in the module register; not standing up.

Module 13 • Treasury & Financial Planning

3 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-13-01	Cash Flow Forecast	SPECIFIED	—	Specified in the module register; not standing up.
APP-13-02	Banking & Reconciliation	SPECIFIED	—	Specified in the module register; not standing up.
APP-13-03	Budget vs Actual	SPECIFIED	—	Specified in the module register; not standing up.

Module 14 • Settlement

3 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-14-01	Payout Cycles	SPECIFIED	—	Specified in the module register; not standing up.
APP-14-02	Fee & Commission Audit	SPECIFIED	—	Specified in the module register; not standing up.
APP-14-03	TCS & TDS Register	SPECIFIED	—	Specified in the module register; not standing up.

Module 15 · E-commerce / OMS

11 app(s) · 2 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP - 15-01	Marketplace OMS	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside <code>npm test</code> — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 15-02	Order Management	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside <code>npm test</code> — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP - 15-03	Manual Data Check	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-04	Reconciliation	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-05	Claims & Disputes	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-06	Returns / RMA	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-07	Channels & Storefronts	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-08	Labels & Documents	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-09	Listing & Catalog Manager	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-10	Size / Fit Recommendation AI	SPECIFIED	—	Specified in the module register; not standing up.
APP - 15-11	AR / Virtual Try-On	SPECIFIED	—	Specified in the module register; not standing up.

Module 16 · HR & Payroll

5 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-16-01	Staff & Contractors	SPECIFIED	—	Specified in the module register; not standing up.
APP-16-02	Time-off & Advances	SPECIFIED	—	Specified in the module register; not standing up.
APP-16-03	Appraisal & Hiring	SPECIFIED	—	Specified in the module register; not standing up.
APP-16-04	Recruitment	SPECIFIED	—	Specified in the module register; not standing up.
APP-16-05	Payout Execution	SPECIFIED	—	Specified in the module register; not standing up.

Module 17 · Marketing

8 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-17-01	Social Calendar	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-02	Campaigns	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-03	Repricing Engine	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-04	Automation	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-05	Blog & Pages	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-06	Events	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-07	Website & Page Builder	SPECIFIED	—	Specified in the module register; not standing up.
APP-17-08	Markdown / Clearance Optimization	SPECIFIED	—	Specified in the module register; not standing up.

Module 18 · AI Content Engine

8 app(s) · 1 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-18-01	Content Engine	SPECIFIED	—	Specified in the module register; not standing up.
APP-18-02	Image Studio	SPECIFIED	—	Specified in the module register; not standing up.
APP-18-03	Video Studio	SPECIFIED	—	Specified in the module register; not standing up.
APP-18-04	Design Studio	SPECIFIED	—	Specified in the module register; not standing up.
APP-18-05	Motion Renderer	TESTED	npm run selftest	Renders a real MP4 and probes it. Command-line only — there is no screen.
APP-18-06	Narration Studio	SPECIFIED	—	Specified in the module register; not standing up.
APP-18-07	Image Generation Slot	SPECIFIED	—	Specified in the module register; not standing up.
APP-18-08	Publisher	SPECIFIED	—	Specified in the module register; not standing up.

Module 19 · SEO, AEO & AIO

3 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-19-01	Technical SEO & Schema	SPECIFIED	—	Specified in the module register; not standing up.
APP-19-02	Answer-Engine Optimization	SPECIFIED	—	Specified in the module register; not standing up.
APP-19-03	AI-Engine Visibility Tracking	SPECIFIED	—	Specified in the module register; not standing up.

Module 20 · Projects & Collaboration

7 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-20-01	Projects & Cases	SPECIFIED	—	Specified in the module register; not standing up.
APP-20-02	Timesheets & Planning	SPECIFIED	—	Specified in the module register; not standing up.
APP-20-03	Approvals	SPECIFIED	—	Specified in the module register; not standing up.
APP-20-04	Forum	SPECIFIED	—	Specified in the module register; not standing up.
APP-20-05	Automation Studio	SPECIFIED	—	Specified in the module register; not standing up.
APP-20-06	Discuss	SPECIFIED	—	Specified in the module register; not standing up.
APP-20-07	Knowledge Base	SPECIFIED	—	Specified in the module register; not standing up.

Module 21 · Dashboard & BI

5 app(s) · 3 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-21-01	CEO Dashboard	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside <code>npm test</code> — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP-21-02	Report Builder	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside <code>npm test</code> — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP-21-03	Group Consolidation	TESTED	npm run apps	Opens in a browser and carries its own self-tests, which now run inside <code>npm test</code> — a broken control turns the suite red. Still a prototype: there is no shared database behind it, so nothing entered is stored anywhere or seen by anyone else. The full click-through of every control runs in its own CI job.
APP-21-04	Excel Dashboard Builder	SPECIFIED	—	Specified in the module register; not standing up.
APP-21-05	ESG / Sustainability Reporting	SPECIFIED	—	Specified in the module register; not standing up.

Module 22 · AI Assistant, Agents & Automation

5 app(s) · 0 above SPECIFIED

ID	App	Status	Proven by	What the rung does not mean
APP-22-01	AI Assistant	SPECIFIED	—	Specified in the module register; not standing up.
APP-22-02	AI Chatbot	SPECIFIED	—	Specified in the module register; not standing up.
APP-22-03	AI Agents	SPECIFIED	—	Specified in the module register; not standing up.
APP-22-04	Agent Guardrails & Run Log	SPECIFIED	—	Specified in the module register; not standing up.
APP-22-05	Knowledge & Retrieval	SPECIFIED	—	Specified in the module register; not standing up.

How to change a row in this table

Not by editing this file — it is generated and overwritten. Edit `brand/site/registry.js`, then run `node brand/site/checkregistry.js`. Raising a row to TESTED requires the command to appear in `docs/verification/EVIDENCE.md` at exit 0, which requires actually running it through `node tools/evidence.js`. There is no path from here to a higher rung that does not go through a command that really ran.

Every technical word above, in plain language

22 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

platform

One piece of software that many separate businesses use at the same time, each seeing only its own information.

Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.

tenant

One business using the platform. Its people, its data and its settings are its own.

Us building mein ek office. Aapka office, aapka saamaan, aapka taala.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

industry pack

A settings file that teaches the system your trade — what you call things, the stages your work moves through, the documents you issue.

Ek hi machine, alag-alag saancha. Saancha badal do, wahi machine doosri cheez banane lagti hai.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

row-level security

A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug.

Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.

integer paise

Money stored as a whole number of paise instead of a decimal, so amounts are exact and rounding can never quietly lose a rupee.

Paisa hamesha poore paise mein ginte hain, aadha-adhoora kabhi nahin — isliye hisaab kabhi ek rupya idhar-udhar nahin hota.

API

The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.

Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

job

One piece of work taken off the queue and done in the background.

Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.

environment

A separate running copy of the system — one for trying things, one that customers actually use.

Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.

deployment

Putting a new version of the software in place so people start using it.

Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.

continuous integration

A robot that checks every change automatically, before anyone can put it live.

Quality-check wala banda gate pe khada. Har maal nikalne se pehle usse guzarta hai.

model

The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.

Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.

fallback

The next option the system automatically moves to when the first one fails or is unavailable.

Bijli gayi toh inverter. Inverter gaya toh mombatti. Andhera kabhi nahin hota.

spend ceiling

A maximum amount the system is allowed to spend on paid services, after which it refuses to spend more instead of warning you.

Jeb mein utne hi paise leke nikle jitna kharch karna hai. Khatam matlab khatam — udhaar nahin.

role

What a person is allowed to see and do — a manager sees more than a counter staff member.

Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.

permission

One specific thing a role is allowed to do, like approving a discount or viewing salaries.

Guchhe ki ek chaabi. Ek chaabi ek darwaza.